Race conditions
Advanced computer architecture

(B-KUL-JLI1QS)

February 29, 2024



Session Date:	February 23, 2024
Partners:	Emma Dejagere
	Brecht Van de Sijpe
Class:	MElsw
Instructor:	Alex Marinsek

1 Introduction

In this lab we got acquainted with atomic operations. An atomic command is executed without getting interrupted. These commands are important to avoid race conditions. Race conditions happen when the behaviour of software depends on the timing of events. We implemented another approach that uses the "semi-parallel" approach.

2 Workflow

This lab existed out of 3 assignments, in which we had to look for the maximum value in an array.

2.1 Nested for-loop on CPU

This was resolved quite easily. Using a for-loop, an array is ran through. Every element is compared to a max_value. If the value of the element is bigger then this value, it is set as the new max_value. This way when the entire array is checked, the max_value will be the largest value of the elements in the array.

2.2 GPU kernel employing atomic operation



To resolve this assignment, a new function is made. The findMaxGPUAtomic()-function calculates the index by multiplying the blockId and the blockDim and adding the threadId. It also calculates the stride by multiplying the blockDim with the gridDim.

After, using a for-loop where i is equal to the index, which was previously calculated. The for-loop runs until i is equal to the length, which will be defined in the main()-function, and every after every run the stride is added to i.
In the for-loop, atomicMax() is used to find the maximum value.

In the main()-function, the array is dynamically allocated using malloc. The memory for the dev_array and dev_maxVal are allocated using cudaMalloc(). Using cudaMemcpy, we copy the array and the maximum value from the host to the device. After this we call the findMaxGPUAtomic()-function. Than the maximum value is copied back to the host.

2.3 GPU kernel employing the "semi-parallel" approach



In this approach we work in layers. Each thread compares a left and right bound and stores the biggest of the two on the index of the left bound. Each new layer the gap between the left and right bound enlarges. The standard version of this implementation works best with array sizes that are a multiple of two.

3 Analysis

When analysing the three methods it is clear that the CPU's computing time rises exponentionally when the array gets bigger. The approach that uses AtomicMax starts with lower computing time but after a specific length is reached the computing time rises exponentionally. The semi-parallel approach starts with the highest computing time of all three implementations, but stays more or less the same. For really big arraysizes this would be beneficial.

If the array length is larger than 2048 the new threads are put in a new block. Since __syncthreads() only synchronizes the threads in the same blocks we have to use cudaDeviceSynchronize() to synchronize the blocks for each layer.

When using much less threads than there are data elements the semi-parallel approach could be modified. You could divide the array into sub-array's so the threads can process all data elements in the sub-array. When all sub-array's are processed only the result of each sub-array has to be compared. For this every result can be put into a new array and this array could be processed in the same way. This principle is showcased in figure 2. A 2D-array could be approached in the same way. It could be a possibility that all the rows are concatenated into one big array and this array is divided in the same way so all available threads are used.

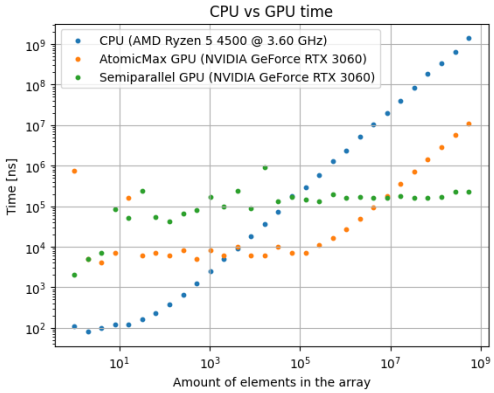


Figure 1: CPU vs GPU time.

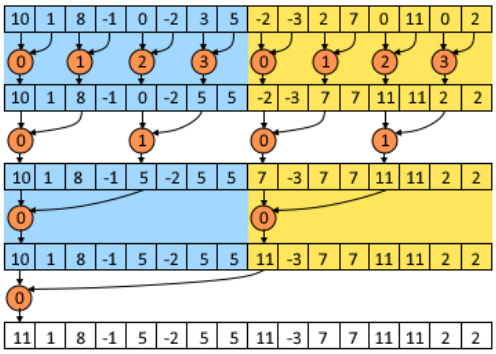


Figure 2: Modified version of semi-parallel

When using much less threads than there are data elements the semi-parallel approach could be modified. You could divide the array into sub-array's so the threads can process all data elements in the sub-array. When all sub-array's are processed only the result of each sub-array has to be compared. For this every result can be put into a new array and this array could be processed in the same way. This principle is showcased in figure 2.

A 2D-array could be approached in the same way. It could be a possibility that all the rows are concatenated into one big array and this array is divided in the same way so all available threads are used.